

RAG-Vorverarbeitung – Module, Ausführung, Validierung

Sechs Dateien für die Vorverarbeitung der RAG-Pipeline: drei Bibliotheksmodule und drei getrennte CLI-Anwendungen. Kein Pytest – die Validierung läuft über direkte Aufrufe.

src/rag_project/loading.py	Dokumente laden	(Bibliothek)
src/rag_project/cleaning.py	Text bereinigen	(Bibliothek)
src/rag_project/chunking.py	Chunks bilden	(Bibliothek)
src/rag_project/main_loading.py	CLI für loading	
src/rag_project/main_cleaning.py	CLI für cleaning	
src/rag_project/main_chunking.py	CLI für chunking	

Die Reihenfolge ist wichtig: laden -> bereinigen -> chunken. Wird zuerst geschunkelt, wandern Steuerzeichen und doppelte Leerzeichen in die Embeddings.

Warum getrennt

Die drei Bibliotheksmodule enthalten keinen ``argparse``-Code, keine Ausgabe auf stdout und kein ``sys.exit``. Sie sind reine Funktionen und Datentypen. Damit sind sie

- importierbar, ohne dass etwas passiert
- aus einem Notebook oder einer späteren Pipeline nutzbar
- einzeln prüfbar, ohne die CLI mitzuziehen

Die ``main\_\*``-Dateien enthalten nur Bedienung: Argumente, Ausgabe, Exit-Codes. Sie importieren die Bibliothek und rufen sie auf.

Die Trennlinie im Überblick:

loading.py	formatregister, loader, metadaten	<- keine Ausgabe
main_loading.py	argparse, print, return 0/1	
cleaning.py	regex, schritte, stats-typen	<- keine Ausgabe
main_cleaning.py	argparse, print, datei schreiben	
chunking.py	splitter, chunk-metadaten, stats	<- keine Ausgabe
main_chunking.py	argparse, print, datei schreiben	

Ausnahme: ``chunking.py`` importiert ``loading`` und ``cleaning`` ****nicht****. Die Pipeline laden -> bereinigen -> chunken zieht ausschließlich ``main\_chunking.py`` zusammen. Die Bibliothek bleibt dadurch frei von Reihenfolge-Wissen.

Ausgabe: wie die Bibliotheken Fortschritt liefern

Die Bibliotheksmodule geben selbst nichts auf stdout aus. Stattdessen nutzen sie zwei Wege -- beide ändern die Eigenschaft "keine Ausgabe" nicht, sie machen Ausgabe nur möglich:

1) on_event-Callback

Jede Funktion mit mehreren Arbeitsschritten nimmt einen optionalen Parameter ``on\_event``. Das ist eine Funktion, die fertige Textzeilen bekommt:

```
from rag_project.loading import load_documents
load_documents(["handbuch.txt"], on_event=print)          # auf stdout
load_documents(["handbuch.txt"], on_event=lambda s: print(s,
file=sys.stderr))
```

Ohne Callback passiert nichts -- die Bibliothek bleibt still.

Funktionen mit on_event:

```
loading.load_document(path, encoding, on_event)
loading.load_documents(paths, encoding, on_event)
cleaning.clean_documents(docs, drop_empty, on_event, *options)
chunking.split_documents(docs, ..., on_event)
chunking.compare_strategies(docs, ..., on_event)
```

2) format_*-Funktionen

Funktionen, die einen fertigen Ausgabetext zurückgeben, statt ihn zu drucken. Damit ist die Formatierung testbar und die Ausgabe entscheidet die CLI:

loading.format_load_report(reports)	Tabelle der geladenen Dokumente
loading.format_document_metadata(doc)	Metadaten eines Dokuments
cleaning.format_cleaning_stats(stats)	Bilanz der Bereinigung
cleaning.format_cleaning_report(b, a)	Wirkung je Bereinigungsschritt
chunking.format_chunking_stats(stats)	Kennzahlen mehrzeilig
chunking.format_strategy_comparison(res)	Strategievergleich als Tabelle
chunking.format_chunk_preview(chunks, n)	erste N Chunks

Beispiel:

```
from rag_project.chunking import split_documents, format_chunking_stats
chunks, stats = split_documents(docs, on_event=print)
print(format_chunking_stats(stats))
```

Fortschritt in der CLI

Jede der drei ``main\_\*``-Dateien hat einen ``--progress``-Schalter. Ist er gesetzt, werden die Fortschrittszeilen nach ``stderr`` geschrieben, die Ergebnisse weiter nach stdout. So bleibt die Ausgabe weiterleitbar:

```
uv run python -m rag_project.main_chunking handbuch.txt --progress >
ergebnis.txt
```

Die Fortschrittszeilen stehen dann im Terminal, die Kennzahlen in der Datei.

Projektstruktur

So sieht das Projekt nach dem Import der sechs Dateien aus. Die neuen Dateien sind mit (NEU) markiert:

RAG-projects/	
├── .env	<- API-Keys (nicht ins Repo)
├── .env.example	Vorlage ohne echte Keys
├── .gitignore	
├── pyproject.toml	Abhängigkeiten, Einstiegspunkte
├── uv.lock	
├── README.md	
├── handbuch.txt	Quelldatei für die RAG-Pipeline
├── src/	
│ └── rag_project/	
│ ├── __init__.py	Public API, main()
│ ├── config.py	.env laden, Settings, SecretStr
│ ├── models.py	Modellkatalog, ModelRegistry
│ ├── providers.py	build_llm, resolve_target
│ ├── main.py	CLI: --check --models --show-config
│ ├── main_chatbot.py	Terminal-Chatbot, /mode-Befehl
│ ├── loading.py	(NEU) Dokumente laden
│ ├── cleaning.py	(NEU) Textbereinigung
│ ├── chunking.py	(NEU) Chunks bilden
│ ├── main_loading.py	(NEU) CLI für loading
│ ├── main_cleaning.py	(NEU) CLI für cleaning
│ └── main_chunking.py	(NEU) CLI für chunking
├── tests/	
│ └── test_config_and_models.py	
├── scratch/	Zwischenergebnisse
│ ├── clean.txt	optional: bereinigter Text
│ └── chunks.txt	optional: Chunk-Ausgabe

Der Import-Pfad ist immer ``rag_project.<modul>``, nie ``src.rag_project.<modul>``.
Das Projekt ist mit dem uv_build-Backend als Paket installiert; ``uv sync``
macht ``src/`` auffindbar.

loading.py (Bibliothek)

Unterstützte Formate: `.txt` `.md` `.markdown` `.rst` (TextLoader), `.pdf` (PyPDFLoader)

Regeln:

- Eine PDF liefert pro Seite ein Document, eine Textdatei genau eines.
- Jedes Document bekommt `source`, `file_name`, `format`, `size_bytes`, `page_index`.
- TextLoader läuft mit `autodetect_encoding=True`: Handbücher aus Windows-Programmen sind oft nicht UTF-8.
- Eine kaputte Datei bricht den Durchlauf ab, statt still übersprungen zu werden. Bei RAG ist eine fehlende Quelle schlimmer als ein Abbruch.

Fehlertypen:

<code>SourceNotFoundError</code>	Datei existiert nicht
<code>UnsupportedFormatError</code>	Endung nicht unterstützt

Öffentliche Funktionen:

<code>detect_format(path)</code>	-> <code>"text" "pdf"</code>
----------------------------------	--------------------------------

load_document(path, encoding)	-> list[Document]
load_documents(paths, encoding)	-> list[Document]
describe(documents)	-> list[LoadReport]

Typ:

LoadReport path, format, documents, characters; .summary() für Ausgaben

 ## main_loading.py (CLI)

```
uv run python -m rag_project.main_loading --help
uv run python -m rag_project.main_loading handbuch.txt
uv run python -m rag_project.main_loading handbuch.txt --preview 400
uv run python -m rag_project.main_loading a.txt b.pdf --preview 0
uv run python -m rag_project.main_loading handbuch.txt --quiet
```

Optionen:

paths	eine oder mehrere Quelldateien
--preview N	Zeichen der Vorschau (0 = keine)
--encoding NAME	Kodierung für Textdateien (Standard: utf-8)
--quiet	nur die Zusammenfassung

Ohne Pfadangabe verwendet die CLI handbuch.txt im aktuellen Verzeichnis, falls vorhanden. Sonst gibt sie die Hilfe aus.

 ## cleaning.py (Bibliothek)

Fünf Schritte, in dieser Reihenfolge:

1 Steuerzeichen entfernen	\\x00-\\x08 \\x0b \\x0c \\x0e-\\x1f \\x7f
unsichtbare Zeichen entfernen	\\u200b-\\u200f \\u202a-\\u202e \\ufe0f
weiches Trennzeichen entfernen	\\u00ad
2 Unicode normalisieren	NFKC (Ligatur "fi" -> "fi")
3 Silbentrennung auflösen	"Anmelde-\\nversuche" -> "Anmeldeversuche"
4 Aufzählungszeichen vereinheitlichen	"•" "■" "。" "、" "。" -> "- "
5 Leerraum normalisieren	NBSP -> Leerzeichen,
Mehrfach-Leerzeichen,	3+ Leerzeilen -> 2, Rand trimmen

Schritt 3 ist der wichtigste für die spätere Suche: Ohne ihn zerbricht ein Wort wie "Anmeldeversuche" in zwei Teile und ist nicht mehr auffindbar.

Öffentliche Funktionen (Bausteine, einzeln nutzbar):

strip_control_characters(text)	Schritt 1
normalize_unicode(text, form)	Schritt 2
join_hyphenated_linebreaks(text)	Schritt 3
normalize_bullets(text, marker)	Schritt 4

<code>normalize_whitespace(text)</code>	Schritt 5
<code>collapse_to_single_line(text)</code>	Hilfsfunktion (Titel, Metadaten)

Öffentliche Funktionen (Gesamtlauf):

<code>clean_text(text, **optionen)</code>	alle Schritte zusammen
<code>clean_document(doc, drop_empty)</code>	auf einem Document
<code>clean_documents(docs, drop_empty)</code>	-> (Liste, CleaningStats)
<code>join_cleaned_text(docs, separator)</code>	Chunks/Dokumente zu einem Text

Optionen von `clean_text`:

<code>unicode_form="NFKC"</code>	NFC NFKC None
<code>normalize_unicode_text=True</code>	Unicode-Normalisierung an/aus
<code>join_hyphens=True</code>	Silbentrennung auflösen
<code>unify_bullets=True</code>	Aufzählungszeichen vereinheitlichen

Typ:

CleaningStats	documents, characters_before, characters_after, empty_after; .removed, .removed_percent, .summary()
---------------	--

main_cleaning.py (CLI)

```
uv run python -m rag_project.main_cleaning --help
uv run python -m rag_project.main_cleaning handbuch.txt
uv run python -m rag_project.main_cleaning handbuch.txt --preview 500
uv run python -m rag_project.main_cleaning handbuch.txt --no-unicode
uv run python -m rag_project.main_cleaning handbuch.txt --no-hyphens
uv run python -m rag_project.main_cleaning handbuch.txt --no-bullets
uv run python -m rag_project.main_cleaning handbuch.txt --keep-empty
uv run python -m rag_project.main_cleaning handbuch.txt --write
scratch/clean.txt
```

Optionen:

<code>paths</code>	eine oder mehrere Quelldateien
<code>--no-unicode</code>	Unicode-Normalisierung abschalten
<code>--no-hyphens</code>	Silbentrennung am Zeilenende nicht auflösen
<code>--no-bullets</code>	Aufzählungszeichen nicht vereinheitlichen
<code>--keep-empty</code>	leere Dokumente behalten statt verwerfen
<code>--preview N</code>	Zeichen der Vorschau (0 = keine)
<code>--write DATEI</code>	bereinigten Text speichern

Die CLI lädt über `main_loading/loading` und bereinigt dann – sie zeigt zuerst, wie viele Zeichen entfernt wurden, dann eine Vorschau.

chunking.py (Bibliothek)

Drei Strategien:

<code>recursive</code>	<code>RecursiveCharacterTextSplitter</code> -- zerlegt an einer Hierarchie
------------------------	--

von Trennzeichen und fällt weiter herunter. Für Fließtext die beste Wahl, weil Absätze erhalten bleiben.

character	CharacterTextSplitter -- harter Schnitt nach Zeichenzahl. Simpel und vorhersagbar, zerlegt aber Sätze mitten durch.
token	TokenTextSplitter -- rechnet in Tokens statt Zeichen. Nützlich, wenn das Kontextfenster des Modells exakt ausgereizt werden soll.

Trennzeichen-Reihenfolge (HANDBOOK_SEPARATORS):

"\\n\\n"	Absatz
"\\n- "	Aufzählungspunkt
"\\n"	Zeilenumbruch
". "	Satzende
" "	Wortgrenze
""	harter Schnitt

Die Voreinstellung von LangChain ist ("\\n\\n", "\\n", " ", ""). Für Handbücher ist die Reihenfolge oben bewusst anders – eine Kapitelüberschrift ist eine natürlichere Grenze als der nächste Zeilenumbruch.

Metadaten je Chunk:

chunk_index	laufende Nummer
chunk_chars	Länge des Chunks
chapter	Kapitelnummer, z. B. "4"
section	Abschnittsnummer, z. B. "4.1"
title	Überschrift, auf 120 Zeichen gekürzt
+ alle Felder aus loading.py (source, file_name, format, ...)	

Die Kapitelerkennung arbeitet auf den ersten drei Zeilen des Chunks. Beginnt ein Chunk mit einer Überschrift, wird die Nummer übernommen; sonst erbt er die des Vorgängers. Das ist die Grundlage für Zitate wie "[4.1]".

Öffentliche Funktionen:

build_splitter(strategy, chunk_size, chunk_overlap, separators)	
detect_heading(text)	-> (kapitel, abschnitt)
extract_title(text)	-> lesbare Überschrift
enrich_chunk_metadata(chunks)	-> list[Document]
split_documents(docs, strategy, chunk_size, chunk_overlap, separators, on_event)	
	-> (Liste, ChunkingStats)
compare_strategies(docs, chunk_size, chunk_overlap, on_event)	->
list[ChunkingStats]	
chunks_to_markdown(chunks)	-> String für die Dateiausgabe

Ausgabe-Helfer (liefern Text, geben nichts aus):

format_chunking_stats(stats)	Kennzahlen mehrzeilig
format_strategy_comparison(results)	Vergleich als Tabelle
format_chunk_preview(chunks, limit)	erste N Chunks mit Metadaten

Konstanten:

STRATEGIES	("recursive", "character", "token")
HANDBOOK_SEPARATORS	Trennzeichen für Handbücher
DEFAULT_SEPARATORS	LangChain-Standardreihenfolge

Typ:

ChunkingStats strategy, chunk_size, overlap, source_documents, chunks,
total_chars, min_chars, median_chars, mean_chars, max_chars,
oversized; .summary_lines() für Ausgaben, .summary()

einzeilig

Fehler:

ValueError chunk_overlap >= chunk_size
UnknownStrategyError Strategie nicht in STRATEGIES

```
-----  
## main_chunking.py (CLI)  
-----
```

```
uv run python -m rag_project.main_chunking --help  
uv run python -m rag_project.main_chunking handbuch.txt  
uv run python -m rag_project.main_chunking handbuch.txt --handbook  
uv run python -m rag_project.main_chunking handbuch.txt --chunk-size 500  
--chunk-overlap 80  
uv run python -m rag_project.main_chunking handbuch.txt --strategy character  
uv run python -m rag_project.main_chunking handbuch.txt --strategy token  
uv run python -m rag_project.main_chunking handbuch.txt --compare  
uv run python -m rag_project.main_chunking handbuch.txt --handbook --show 5  
uv run python -m rag_project.main_chunking handbuch.txt --write  
scratch/chunks.txt
```

Optionen:

paths	eine oder mehrere Quelldateien
--strategy NAME	recursive (Standard) character token
--chunk-size N	Zielgröße (Standard: 800)
--chunk-overlap N	Überlappung (Standard: 100)
--handbook	Handbuch-Trennzeichen verwenden
--compare	alle Strategien bei gleichem Zielwert vergleichen
--show N	erste N Chunks anzeigen (0 = keine)
--write DATEI	Chunks als Markdown speichern

Diese CLI fährt die vollständige Pipeline: laden -> bereinigen -> chunken.
Sie zeigt zuerst, wie viele Dokumente geladen und wie viele Zeichen durch die
Bereinigung entfernt wurden, dann die Chunk-Kennzahlen.

```
-----  
## Wo die Quelldokumente liegen  
-----
```

Der Pfad ist zentral in der .env konfiguriert -- nicht im Code. Die Auflösung
liegt in rag_project/paths.py, damit die drei CLIs nicht drei leicht
abweichende Varianten haben.

```
# .env  
DOCS_DIR=docs # relativ zur Projektwurzel  
DEFAULT_DOCUMENT=handbuch.txt
```

```
DOCUMENT_FILES=                                # optional: a.txt,b.md
```

Bei dieser Verzeichnisstruktur:

```
RAG-projects/
├── docs/
│   └── handbuch.txt      <- die Quelldatei
├── src/
│   └── rag_project/
└── tests/
```

Auflösungspriorität:

1. Pfade auf der Kommandozeile
2. DOCUMENT_FILES aus der .env (kommagetrennte Liste)
3. DOCS_DIR / DEFAULT_DOCUMENT aus der .env
4. Fallback: docs/handbuch.txt relativ zur Projektwurzel

Wichtig: Pfade von der Kommandozeile werden gegen das *Arbeitsverzeichnis* aufgelöst, konfigurierte Pfade gegen die *Projektwurzel*. So verhält sich "uv run python -m rag_project.main_loading docs/handbuch.txt" wie erwartet, während DOCS_DIR=docs unabhängig vom Startordner funktioniert.

Pfade prüfen

Alle drei CLIs haben --show-paths:

```
uv run python -m rag_project.main_chunking --show-paths
```

Ausgabe:

```
1 Dokumentpfad(e):
  [ok  ] C:\AZAV-ML\RAG-projects\docs\handbuch.txt (9476 Bytes)
```

Wenn nichts gefunden wird

Die Fehlermeldung nennt die aktuelle Konfiguration, nicht nur den fehlenden Dateinamen:

```
Fehler: Kein Quelldokument gefunden. Gesucht in:
C:\AZAV-ML\RAG-projects\docs\handbuch.txt
```

Aktuelle Konfiguration:

```
PROJECT_ROOT      : C:\AZAV-ML\RAG-projects
DOCS_DIR          : docs
DEFAULT_DOCUMENT  : handbuch.txt
erwarteter Pfad   : C:\AZAV-ML\RAG-projects\docs\handbuch.txt
```

Lege die Datei dorthin oder setze DOCS_DIR in der .env.

Andere Dateien verwenden

```
# eine andere Datei, explizit
uv run python -m rag_project.main_chunking docs/anhang.md --handbook
```

```
# dauerhaft eine andere Datei
# .env: DEFAULT_DOCUMENT=anhang.md

# mehrere Dateien dauerhaft
# .env: DOCUMENT_FILES=handbuch.txt,anhang.md
```

paths.py (Bibliothek)

```
resolve_documents(paths, settings)  -> list[Path]
describe_paths(paths)               -> String für die CLI-Ausgabe
```

```
DocumentPathError  kein Quelldokument auffindbar
FALLBACK_PATH      docs/handbuch.txt relativ zur Projektwurzel
```

Neue Felder in config.py

```
docs_dir: Path = Path("docs")
default_document: str = "handbuch.txt"
document_files: str = ""
```

```
settings.docs_path      absoluter Pfad des Ordners
settings.document_path  absoluter Pfad des Standarddokuments
settings.document_paths  Liste aller zu ladenden Dateien
```

PROJECT_ROOT sucht aufwärts nach pyproject.toml, statt Ebenen zu zählen --
damit bricht der Pfad nicht, wenn sich config.py im Baum verschiebt.

Wie der User andere Dateien angibt

handbuch.txt ist KEIN fest verdrahteter Pfad, sondern nur der Rückfallwert,
wenn kein Argument angegeben wird. Die Zeile in jeder main_*.py:

```
DEFAULT_SOURCE = "handbuch.txt"
```

Und die Prüfung davor:

```
paths: list[str] = list(args.paths)
if not paths and Path(DEFAULT_SOURCE).is_file():
    paths = [DEFAULT_SOURCE]
```

Gibt es handbuch.txt im aktuellen Verzeichnis nicht, zeigt die CLI die Hilfe.
Der Rückfall greift also nur, wenn die Datei existiert.

Eine andere Datei angeben

```
uv run python -m rag_project.main_loading mein_dokument.txt
uv run python -m rag_project.main_cleaning mein_dokument.txt
uv run python -m rag_project.main_chunking mein_dokument.txt --handbook
```

Weil paths als nargs="*" deklariert ist, gehen auch mehrere Dateien in einem Aufruf:

```
uv run python -m rag_project.main_loading a.txt b.txt c.md
uv run python -m rag_project.main_chunking handbuch.txt handbuch_teil2.txt
--handbook
```

Ein Pfad in einem anderen Verzeichnis

Relative und absolute Pfade funktionieren beide:

```
# Unterordner
uv run python -m rag_project.main_chunking docs/handbuch.txt --handbook

# eine Ebene hoch
uv run python -m rag_project.main_chunking ../daten/handbuch.txt

# absolut (Windows)
uv run python -m rag_project.main_chunking
"C:\AZAV-ML\RAG-projects\daten\handbuch.txt"

# absolut (macOS/Linux)
uv run python -m rag_project.main_chunking /home/user/daten/handbuch.txt
```

Bei Windows-Pfaden mit Leerzeichen sind Anführungszeichen nötig.

Alle Dateien eines Ordners

Die Shell löst das Muster auf -- auf Linux/macOS:

```
uv run python -m rag_project.main_chunking daten/*.txt --handbook
```

Auf Windows PowerShell:

```
uv run python -m rag_project.main_chunking (Get-ChildItem
daten\*.txt).FullName --handbook
```

Was ohne Argument passiert

Situation	Ergebnis
-----	-----
handbuch.txt im aktuellen Verzeichnis	wird verwendet, mit Hinweiszeile
handbuch.txt fehlt	Hilfe wird ausgegeben, Exit-Code 0
Pfad angegeben, Datei fehlt	"Fehler: Source not found: ...",
	Exit-Code 1

Die Fehlermeldung nennt den vollständigen Pfad, den die CLI gesucht hat -- damit ist sofort sichtbar, ob das Arbeitsverzeichnis stimmt.

Nur unterstützte Formate

.txt .md .markdown .rst und .pdf

Bei einer anderen Endung:

Fehler: Unsupported file format: '.docx' (datei.docx).

Unterstützt: .md, .markdown, .pdf, .rst, .txt

Ein Word-Dokument muss also vorher als PDF oder Text exportiert werden. Für .docx direkt gibt es in langchain-community den Docx2txtLoader -- das wäre eine Erweiterung des Formatregisters in loading.py.

Dauerhaft ein anderes Dokument

Soll das Projekt künftig mit einer anderen Datei arbeiten, ändere eine Zeile in jeder der drei main_*.py:

```
DEFAULT_SOURCE = "daten/handbuch_2026.txt"
```

Sauberer wäre, den Default aus der Umgebung zu lesen -- etwa DOCUMENT_PATH in der .env. Das passt zur bestehenden Konfigurationsschicht und wäre eine kleine Ergänzung in config.py.

Validierung

Kein Pytest. Geprüft werden die Module über direkte Aufrufe im Terminal.

1) Syntax- und Importprüfung (ohne API, ohne Netzwerk):

```
uv run python -c "import rag_project.loading, rag_project.cleaning,  
rag_project.chunking; print('Bibliotheken ok')"
```

```
uv run python -c "import rag_project.main_loading,  
rag_project.main_cleaning, rag_project.main_chunking; print('CLIs ok')"
```

2) Die drei CLI-Anwendungen:

```
uv run python -m rag_project.main_loading --help
```

```
uv run python -m rag_project.main_cleaning --help
```

```
uv run python -m rag_project.main_chunking --help
```

3) Fehlerfälle:

```
uv run python -c "  
from pathlib import Path  
from rag_project.loading import load_document, SourceNotFoundError,  
detect_format, UnsupportedFormatError  
from rag_project.chunking import split_documents, build_splitter,  
UnknownStrategyError  
try: load_document('gibt-es-nicht.txt')  
except SourceNotFoundError as e: print('ok:', e)
```

```

try: detect_format(Path('a.docx'))
except UnsupportedFormatError as e: print('ok:', e)
try: split_documents([], chunk_size=100, chunk_overlap=100)
except ValueError as e: print('ok:', e)
try: build_splitter('magic')
except UnknownStrategyError as e: print('ok:', e)
"

```

4) Reinigungsfunktionen einzeln:

```

uv run python -c "
from rag_project.cleaning import clean_text
print(repr(clean_text('Anmelde-\nversuche')))
print(repr(clean_text('a    b\n\n\n\n\nnc'))))
"

```

5) Vollständige Pipeline auf dem Handbuch:

```
uv run python -m rag_project.main_chunking handbuch.txt --handbook --show 3
```

6) Strategievergleich:

```
uv run python -m rag_project.main_chunking handbuch.txt --compare
```

Ergebnis der bisherigen Validierung (Handbuch, 9343 Zeichen):

```

12 von 12 Prüfungen bestanden
Bereinigung      9343 -> 9342 Zeichen, alle 10 Kapitel intakt
Chunking         18 Chunks | min 315 / median 729 / max 794 | übergroß 0
Metadaten        Kapitel in 17 von 18 Chunks, Abschnitt in 8 von 18
Überlappung      23 von 23 benachbarten Paaren teilen Text
Kapitelfolge     ,1,1,2,2,3,3,4,4,6,6,7,7,7,9,9,10,10

```

ANMERKUNG: Diese Validierung lief als Nachbildung der dokumentierten Semantik in einer JavaScript-Umgebung, nicht als echter Python-Import der Module. Geprüft sind damit die Regeln (Regex, Schritte, Reihenfolgen, Fehlerfälle) und die tatsächlichen Zahlen des Handbuchs. Die erste echte Ausführung in Python ersetzt Punkt 1 bis 3 – sie ist der eigentliche Beweis.

Das Ergebnis bestätigt zugleich eine Korrektur der ersten Schätzung: Bei 800/100 wurden zunächst 14 Chunks aus der Zeichenzahl hochgerechnet, tatsächlich sind es 18. Die Handbuch-Trennzeichen schneiden feiner als die Standardreihenfolge, weil sie an Aufzählungspunkten und Satzenden zusätzlich trennen.

Was als Nächstes fehlt

indexing.py	Embeddings + Vektorindex (fastembed + qdrant-client sind in pyproject.toml bereits vorhanden)
retrieval.py	Dense-Suche, BM25 (rank-bm25 ist vorhanden) und Fusion
generation.py	Antwort aus den gefundenen Chunks mit Zitaten

evaluation.py recall@k gegen die zehn Unterabschnittsfragen

Für retrieval.py ist die BM25-Komponente der wichtigste offene Punkt: Die Zahlen im Handbuch (15 Minuten, 30 Minuten, 24 Stunden, 5 Jahre, 10 MB, 30 Tage, 14 Tage, 20 Minuten) sind genau der Fall, bei dem Embeddings unscharf werden und exakte Suche gewinnt.

Dateigrößen

Bibliothek		
src/rag_project/loading.py	~5 KB	laden, Formatregister,
Metadaten		
src/rag_project/cleaning.py	~8 KB	fünf Schritte, Statistiken
src/rag_project/chunking.py	~12 KB	drei Strategien,
Chunk-Metadaten		
CLI		
src/rag_project/main_loading.py	~3 KB	Argumente, Ausgabe
src/rag_project/main_cleaning.py	~3 KB	Argumente, Ausgabe,
Dateischreiben		
src/rag_project/main_chunking.py	~5 KB	Pipeline, Vergleich,
Dateischreiben		